

SECTION 10

Portfolio Chatbot

Detailed Student Instructions | Capstone

PROJECT GOAL

Present student projects and answer portfolio questions with a safe local rule-based chatbot.

What you will learn

- Connect index.html, style.css, and script.js.
- Personalize text, colors, and a parent- or teacher-approved picture.
- Modify and test the JavaScript feature for this project.
- Upload the correct folder contents and publish with GitHub Pages.
- Submit both the repository link and the published webpage link.

Files you must keep together

PROJECT ROOT	PURPOSE
index.html	Structure and visible content
style.css	Colors, spacing, and responsive layout
script.js	Interaction and app logic
images/	Profile and project pictures
README.md	Short reference inside the code folder

Repository name used in this guide: **10-portfolio-chatbot**

Part A - Open and Personalize the Project

1

Find the correct project folder

Open the downloaded course package and locate **10-portfolio-chatbot**. Work on a copy so the original starter remains available.

2

Preview the starter webpage

Double-click **index.html**. It should open in a browser. Keep this browser tab open; after every change, save the file and refresh the tab.

3

Open the code files

Use Visual Studio Code, Notepad, TextEdit in plain-text mode, or another code editor. Open **index.html**, **style.css**, and **script.js**.

Edit index.html

Replace all sample identity text

Change Jordan, the biography, interests, and project descriptions. Use only parent- or teacher-approved public information.

Replace the project cards

Each card should include a project title, one concise description, and optionally a link to another published GitHub Pages project.

Change chatbot labels

Update Ask about Jordan and the greeting so the displayed name is consistent everywhere.

Replace the picture

The starter path is **images/profile-placeholder.svg**. Choose a JPG or PNG that the student has permission to publish. Rename it with lowercase letters and no spaces, place it inside the images folder, and change the path to **images/profile.jpg**.

```

```

Important: If Windows shows **profile.jpg.jpg**, turn on file extensions and rename it to **profile.jpg**. GitHub treats uppercase and lowercase filenames as different.

Part B - Customize CSS and JavaScript

1. Change the visual design

Customize the portfolio accent colors, chat bubbles, and project cards. Keep user and bot bubbles visually different.

```
:root {
  --bg: #07111f;
  --ink: #f4f7fb;
  --accent: #55d6be;
  --accent2: #ffb45e;
}
```

2. Understand the project logic

Edit the responses array. Each response needs keywords and one answer. Use only public portfolio facts. This project is rule-based: it does not call an AI service, does not require an API key, and does not send chat text to the internet.

```
{
  keywords: ['robot', 'engineering'],
  answer: 'I built a sensor-based robot project.'
}
```

3. Make one meaningful JavaScript change

Changing only a color is not a JavaScript change. Change sample data, text produced by the program, a formula, a setting, a response, or a feature. Save script.js and test the result.

Extension challenge

Add project links and a new response that directs visitors to the correct project card.

Privacy and safety check

- Use only approved pictures and general public information.
- Do not publish home addresses, private phone numbers, student IDs, passwords, API keys, or private schedules.
- This starter project does not need an API key. Never paste a secret key into JavaScript.

Part C - Test Before Uploading

Complete every test below. If one test fails, fix it before creating the GitHub repository.

- ☐ Greeting appears when the page opens.
- ☐ Every suggested question receives a correct answer.
- ☐ New keywords trigger the intended answer.
- ☐ An unknown question displays the safe fallback.
- ☐ Clear removes the conversation and restores the greeting.

General browser checks

- ☐ index.html opens without a blank page.
- ☐ Text is readable and no content overlaps.
- ☐ The page works when the browser is narrow like a phone.
- ☐ Every picture loads; no broken-image icon appears.
- ☐ Refreshing the page does not create an error.

Confirm the upload structure

```
10-portfolio-chatbot/  
|-- index.html  
|-- style.css  
|-- script.js  
|-- README.md  
`-- images/  
    |-- profile.jpg
```

The most common mistake

Do not create an extra nested folder such as **10-portfolio-chatbot/10-portfolio-chatbot/index.html**. When the repository opens, you should immediately see index.html in the file list.

Choose the final repository name now

Use **10-portfolio-chatbot**. Lowercase letters and hyphens make the published address easy to type.

Part D - Upload to GitHub and Publish

- 1 Sign in**
 Go to github.com and sign in. Use the account approved for the class.
- 2 Create a new repository**
 Select the plus sign in the upper-right area and choose **New repository**. Enter **10-portfolio-chatbot** as the repository name.
- 3 Choose repository settings**
 Choose **Public** for the simplest GitHub Pages workflow on a free account. Optionally select Add a README file. Click **Create repository**.
- 4 Open the upload screen**
 Above the repository file list, choose **Add file**, then **Upload files**.
- 5 Upload the project contents**
 Drag index.html, style.css, script.js, README.md, and the images folder into the upload area. Wait until every filename appears.
- 6 Commit the upload**
 Enter the message **Add Portfolio Chatbot website**. Commit the changes to the main branch.
- 7 Open Pages settings**
 Choose **Settings**. In the left sidebar under Code and automation, choose **Pages**.
- 8 Select the publishing source**
 Under Build and deployment, set Source to **Deploy from a branch**. Choose branch **main** and folder **/ (root)**. Click **Save**.
- 9 Open the site**
 Return to Settings - Pages after the deployment finishes. Open the displayed site link, or use the address shown below.

<https://YOUR-USERNAME.github.io/10-portfolio-chatbot/>

Replace YOUR-USERNAME with the exact GitHub username. Project-page addresses must include the repository name and the final slash.

Part E - Update, Troubleshoot, and Submit

How to update the website later

1

Open the repository

Select the file you need to update, such as index.html, style.css, or script.js.

2

Edit or replace the file

For a text edit, use the pencil icon. For a local replacement, use Add file - Upload files and upload the changed file with the same name.

3

Commit the change

Write a short message describing the change and commit it. GitHub Pages will deploy the updated main branch.

4

Force-refresh the site

If the old version remains, press Ctrl+Shift+R on Windows or Command+Shift+R on Mac.

Troubleshooting

PROBLEM	CHECK
404 page	Pages uses main and / (root); index.html is at the root; URL includes repository name.
No design	Filename is exactly style.css and HTML contains href="style.css".
Buttons fail	Filename is exactly script.js; HTML contains src="script.js"; inspect first console error.
Picture fails	Image exists in images/; spelling, capitals, and extension match the src path.
Site not updated	Confirm the commit is on main; check the Pages deployment; force-refresh.

Final submission checklist

- ☐ Repository link: <https://github.com/USERNAME/REPOSITORY>
- ☐ Published webpage link: <https://USERNAME.github.io/REPOSITORY/>
- ☐ Student name and project section clearly identified for the teacher
- ☐ All required customizations and tests completed

Official GitHub references

[Adding a file to a repository](#)

[Configuring a publishing source for GitHub Pages](#)

[Troubleshooting GitHub Pages 404 errors](#)

[What is GitHub Pages?](#)